

OpenFrame HiDBガイド

OpenFrame/HiDB 7.2



Copyright © 2023 TmaxSoft Co., Ltd. All Rights Reserved.

文書情報

文書名: OpenFrame HiDBガイド

発行日: 2023年12月29日

ソフトウェアバージョン: OpenFrame/HiDB 7.2

ガイドバージョン: v3.1.2

Website

<http://www.tmaxsoft.com>

Copyright Notice

Copyright © 2023 TmaxSoft Co., Ltd. All Rights Reserved.

Restricted Rights Legend

このソフトウェア（Tmax OpenFrame®）マニュアルの内容とプログラムは、日本国の著作権法および国際条約によって保護されています。マニュアルの内容とプログラムは、TmaxSoft Co., Ltd.との使用許諾契約書の下でのみ使用することができ、マニュアルは使用許諾契約で許可されている範囲を除いては、配布または複製することができません。TmaxSoftの書面による事前の承諾を得ることなく、このマニュアルの全部または一部を電子的または機械的な方法を問わず、転送、複製、配布したり、または二次的著作物を作成する等の行為を一切禁じます。

このソフトウェアのマニュアルとプログラムの使用許諾契約は、いかなる場合においても、マニュアル及びプログラムと関連する知的財産権（登録の有無を問わず）を譲渡するものと解釈されず、TmaxSoftのブランド、ロゴ、商標等の使用権限を与えるものではありません。このマニュアルは、情報を提供する目的でのみ提供しており、これに伴う契約上の直接的ないしは間接的な責任を負わず、マニュアルの内容は法律上もしくは商業的な特定の条件が満たされることを保証しません。マニュアルの内容は、製品のアップグレード及び修正により、その内容が予告なく変更されることがあり、内容上の誤りがないことを保証しません。

Trademarks

Tmax®, Tmax Tiberio®, Tmax OpenFrame®は、TmaxSoft Co., Ltd.の登録商標です。その他、記載されている会社名、製品名などは、各社の商標または登録商標です。

Open Source Software Notice

この製品には、OpenSSL、RSA Data Security, Inc.、Apache FoundationおよびJean-loup Gaillyと Mark Adlerによって開発またはライセンス取得されたオープンソフトウェアが含まれています。詳細は、製品ディレクトリの\${INSTALL_PATH}/license/oss_licensesに記載されている事項を参照してください。

目次

このガイドについて	vii
第1章 紹介	1
1.1. IMSシステム	1
1.2. データベースの基礎	1
1.3. 階層データベース	2
1.4. アクセス・メソッド	3
1.5. サポート範囲	5
第2章 データベースの管理	7
2.1. 概要	7
2.2. DBD	7
2.3. PSB	10
2.4. ACB	12
2.5. 可変長セグメント	12
第3章 DL/I関数の呼び出し	15
3.1. DL/I関数	15
3.2. DL/I機能	16
3.3. PCBマスク	17
3.4. AIBマスク	20
3.5. SSA	21
第4章 高度な機能	27
4.1. 論理関係(Logical Relationship)	27
4.2. 二次索引(Secondary Indices)	29
第5章 マイグレーション	33
5.1. マイグレーションの手順	33
5.2. DBDの作成	34
5.3. スキーマの作成およびコピーブックの処理	34
5.4. セグメント表の作成	35
5.5. DLIライブラリの作成	36
5.6. 標準データ・フォーマットの変換	36
5.7. コード変換	38
5.8. データのロード	38
付録 A. DL/Iステータス・コード	41
付録 B. データベース定義の固有の構文	49
B.1. SEGM文	49
B.1.1. EXTPROC	49
B.1.2. GNALLPARENT	50
索引	51

図目次

[図 1.1]	階層データ構造	2
[図 2.1]	DBD作成プロセス	8
[図 2.2]	PSB作成プロセス	10
[図 2.3]	ACB作成プロセス	12
[図 3.1]	非修飾SSAの形式	21
[図 3.2]	修飾SSAの形式	22
[図 3.3]	単一コマンド・コードを持つ非修飾SSA形式	23
[図 3.4]	単一コマンド・コードを持つ修飾SSA形式	24
[図 4.1]	2つの物理データベース間の関係	27
[図 4.2]	論理関係にある2つの物理データベース	28
[図 4.3]	2つの付加的な論理データベース	28
[図 4.4]	二次索引データベースを持つデータベース	30
[図 5.1]	OpenFrame/HiDBマイグレーションの手順	33
[図 5.2]	OpenFrame/HiDB標準データ・フォーマット	37

このガイドについて

対象読者

本書は、リホスト・ソリューションのTmax OpenFrame[®](以下、OpenFrame)と階層型データベースのOpenFrame/HiDBを使用するユーザーを対象としています。OpenFrame/HiDBは階層型データベース(Hierarchical Database)または高性能データベース(High Performance Database)の略であり、IBMメインフレームのIMS/DBに対応するOpenFrame製品です。

前提知識

本書を理解するには、OpenFrame/BatchおよびメインフレームのIMSシステムについての知識が必要です。本書を読む前に、メインフレームのIMS関連ガイドを読まれることをお勧めします。

制限事項

現在、OpenFrame/HiDBがサポートしていない機能、オプション、パラメータについての説明は省略しています。

本書の構成

本書は、計6章と付録で構成されています。

各章の主な内容は以下のとおりです。

- 第1章: 紹介

IBMメインフレームのIMSシステム、データベースの基礎、階層的データベース、アクセス・メソッドおよびOpenFrame/HiDBのサポート範囲について説明します。

- 第2章: データベースの管理

DBD、PSB、ACBスクリプトをコーディング、コンパイルして、OpenFrame/HiDBデータベースを実装する方法について説明します。

- 第3章: DL/I関数の呼び出し

DL/Iの機能とPCB Mask、SSA (セグメント検索引数)について説明します。

- 第4章: 高度な機能

OpenFrame/HiDBの高度な機能である論理関係(Logical Relationship)と副次索引(Secondary Indices)について説明します。

- 第5章: マイグレーション

OpenFrame/HiDBのマイグレーションの手順と各段階で実行する作業内容について説明します。

- 付録A: DL/Iステータス・コード

アプリケーション・プログラムでDL/I関数を呼び出した後に返されるステータス・コードのタイプ別のエラーと原因および原因別の対処方法について説明します。

- 付録B: データベース定義の固有の構文

データベースを定義する際に使用されるOpenFrame/HiDB固有の構文について説明します。

表記上の規則

表記	意味
<AaBbCc123>	プログラム・ソースコードのファイル名
<Ctrl>+C	CtrlキーとCキーを同時に押す
[Button]	GUIのボタン、メニュー名
太字	強調
「」、『』（鍵カッコ）	関連文書、あるいはガイド内の他の章および節の表示
「入力項目」	画面UI上の入力項目
ハイパーリンク	メール・アカウント、Webサイト
>	メニューの実行順
+----	サブディレクトリ/ファイル有り
----	サブディレクトリ/ファイル無し
参考	参照/注意事項
[図 1.1]	図の名前
AaBbCc123	コマンド、コマンド実行結果の画面出力、サンプル・コード
{ }	必須パラメータ値
[]	オプション・パラメータ値
	選択パラメータ値

システム要件

	要求事項
プラットフォーム	Solaris 11(SunOS 5.11)以上 (32bit、64bit)
	Linux x86 2.6以上 (32bit、64bit)
	Linux ia64 2.6以上 (32bit、64bit)
ハードウェア	5GB以上のハードディスク
	8GB以上のメモリ
データベース	Tibero 6 FS07
コンパイラー	Micro Focus COBOL、NetCOBOL、OpenFrame COBOL
	OpenFrame PL/I
	OpenFrame ASM
OpenFrame製品群	OpenFrame/Base 7.1

参考

IBMまたはHP-UXプラットフォームをご使用の場合は、ティーマックスソフトの技術サポート・チームにお問い合わせください。

関連文書

ガイド	説明
OpenFrame スタートガイド	OpenFrameソリューションの概説とOpenFrameの全ガイドについて説明しています。
OpenFrame Baseガイド	OpenFrame/Baseを構成する全般的な機能について説明しています。
OpenFrame Batchガイド	OpenFrame/Batchを構成する全般的な機能について説明しています。
OpenFrame データセットガイド	OpenFrameデータセットを紹介し、データセットの種類やカタログ方法について説明しています。
OpenFrame TSAMガイド	VSAMに対応する製品であるTSAMについて紹介し、データセットの種類、管理方法、処理方法について説明しています。
OpenFrame TJESガイド	TJESを使用してOpenFrameシステムのバッチジョブを管理する方法について説明しています。
OpenFrame ツールリファレンスガイド	OpenFrameシステムの運用に使用する各種ツール・プログラムについて説明しています。
OpenFrame ユーティリティ リファレンスガイド	OpenFrameで提供される多様なユーティリティ・プログラムについて説明しています。

参考文献

製品	ガイド
IBM Mainframe	IMS for the COBOL programmer
	IMS Primer
	IMS V7 Administration Guide: Database Manager
	IMS V7 Administration Guide: System
	IMS V7 Application Programming: Database Manager
	IMS V7 Application Programming: Transaction Manager
	IMS V7 Utilities Reference: System
	IMS V7 Utilities Reference: Database and Transaction Manager

第1章 紹介

本章では、IBMメインフレームのIMSシステムについて簡単に紹介し、OpenFrame/HiDB製品のサポート範囲について説明します。

1.1. IMSシステム

OpenFrame/HiDB(以下、HiDB)は、階層型データベース(Hierarchical Database)または高性能データベース(High Performance Database)の略であり、IBMメインフレームのIMS/DBに対応するOpenFrame製品です。IMSシステムは、1968年に紹介されたIBMメインフレームのソフトウェアであり、大型ビジネス・システムのためのトランザクション管理機能およびデータベース管理機能を提供します。

IMSシステムは、データ通信管理(DC)とデータベース管理(DB)の2つの主要製品で構成されています。

- IMS/DC製品

メッセージ・ベースのトランザクション処理システムです。トランザクションとは、特定のアプリケーションの実行に必要な入力データ・パケットです。IMS/DC製品は、ネットワーク・ターミナルからのメッセージや、他のアプリケーションで作成されたトランザクション・メッセージの処理を可能にする基盤サービスを提供します。

- IMS/DB製品

階層データベースの管理システムとして、ユーザーのアプリケーション・データを構造的に格納・管理する機能を提供します。同製品は、IMS/DCで実行されるアプリケーションや純粋なバッチ・アプリケーションからデータベースに容易にアクセスできるインターフェースを提供します。

アプリケーションがIMSシステムで管理されるトランザクション・メッセージやデータベースにアクセスするためには、標準IMS関数を呼び出す必要があります。アプリケーションは、各プログラム言語で提供される標準アプリケーション・インターフェースを利用してIMS関数を使用することができます。一般的にこのようなインターフェースをDL/I(Data Language One)と呼びます。

1.2. データベースの基礎

コンピューター・システムが開発された当初、データは特定のアプリケーションに従属された個別データセットに格納されたり、あるいは個別プログラムの一部として作成されました。そのため、同じ内容のデータが異なるデータセットに重複して格納されることが頻繁に発生しました。たとえば、ユーザーに関する詳細情報は注文プログラムと配送プログラムで同時に使用される必要がありますが、この場合、以下のような問題が発生します。

- データが各プログラムで独立的に処理されるため、データの整合性が取れないことがあります。たとえば、注文プログラムで使用するデータセットのユーザー・アドレスと配送プログラムで使用するデータセットのユーザー・アドレスが異なる場合があります。
- 共通的なデータの内容が変更された場合、複数の場所に格納されているデータの内容も変更される必要があるため、変更範囲が広くなり、処理に長時間を要することになります。
- データを制御する中央集中的な装置がないため、データへの不正なアクセスやデータの流出に対する管理が困難です。
- 重複データが存在するため、ストレージの容量が無駄になります。

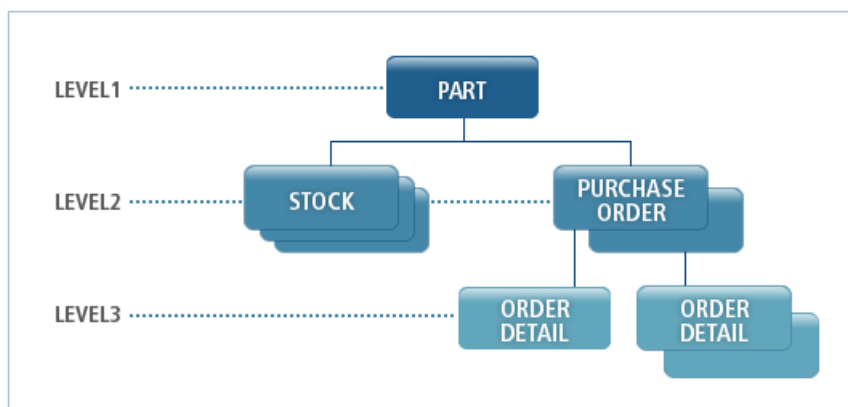
一方、ユーザーのアプリケーション・データを構造的に格納・管理するデータベース管理システムを使用すると、以下のような効果が得られます。

- データの整合性を保ちつつ、複数のアプリケーションが同時にデータにアクセス、変更できるようにします。
- アプリケーションの実行に失敗しても、複数のデータベース・レコードにわたって存在しているアプリケーション・データの整合性を保ちます。
- データのバックアップおよび復旧ユーティリティを提供し、お客様の重要なビジネス・データの消失に備えます。
- データへのモニタリングおよび性能をチューニングできるユーティリティを提供します。

1.3. 階層データベース

IBMのIMS/DBはデータを格納する方式として階層モデル(Hierarchical Model)を使用しています。このモデルで個別のデータ項目は階層構造におけるセグメントとして管理されます。

[図 1.1] 階層データ構造



階層モデルのデータ構造はセグメント・タイプとセグメント・インスタンスに分けて説明できます。セグメント・タイプはセグメントの種類を意味しており、セグメント・インスタンスは、セグメント・タイプの実際のデータ内容です。

階層データ構造の各セグメントは、親子関係で結ばれています。親セグメント・インスタンスは、0または1つ以上の子セグメント・インスタンスを持つことができます。一方、子セグメントは親セグメント・インスタンスを1つのみ持ちます。同じ親セグメント・インスタンスを持つ、同じセグメント・タイプの子セグメント・インスタンスを**ツイン・セグメント**といいます。ただし、同じ親セグメント・インスタンスを持つ異なるセグメント・タイプの子セグメント・インスタンスは**兄弟セグメント**と呼びます。

親セグメントは複数の子セグメント・タイプを持つことができます。また、子セグメントは同時に親セグメントになることもできます。つまり、子セグメントも子セグメントを持つことが可能です。親セグメントを持たないセグメントを**ルート・セグメント**と呼びます。ルート・セグメントは最上位に1つだけ存在します。1つのルート・セグメント・インスタンスの下位に存在するすべてのセグメント・インスタンスをDLIデータベース・レコードといいます。このような、DLIデータベース・レコードをすべて集めたものが**DLIデータベース**です。

DLIデータベース内に格納されている特定のセグメントを検索するためにルート・セグメントから先祖セグメントを連続的に並べた順序を**セグメントの階層パス(Hierarchical Path)**といいます。DLIデータベース内に存在する特定のセグメントを他のセグメントと区別し、かつ直接アクセスするために、DLIは順次フィールドを使用します。一般的に各セグメントには順次フィールドで表示されているフィールドが1つ含まれています。同じ親インスタンスを持つツイン・セグメント・インスタンスは、一意の値を持つ順次フィールドを有する必要があります。

前述したセグメントの階層パス以外に、IMS/DBでは以下のような2つのデータベース・セグメントのアクセスパスを提供します。この2つの方法は、いずれもアプリケーションが物理データベースにアクセスするアクセス・パスを提供します。これらは、定義済みの基本的なデータベースの階層構造上に追加的に定義されて使用されます。

- 論理的関係(Logical Relationship)

1つ以上の物理データベースを論理的に連結して使用できるようにします。アプリケーションは、1つのデータベースを使用しているかのようにアクセスできます。

- 二次索引(Secondary Indices)

1つの物理データベースで、ルート・セグメントまたはルート・セグメント以外の別のセグメントを使用してデータベース・レコードにアクセスできるアクセス・パスを提供します。

1.4. アクセス・メソッド

前述したデータベース、セグメントおよびレコードは、データベースのアクセス・メソッド(HSAM、HDAM、DEDBなど)によって物理的に構成されます。また、データベースのアクセス・メソッドの下部では、物理的なディスクにデータをI/Oするため、メインフレームのデータセット・アクセス・メソッドが使用されます。

階層データベースの構造およびアプリケーションからのアクセス方式は、DBD、PSB、ACBなどの制御ブロックによって定義されます。制御ブロックは、データベース管理者によってマクロ・スクリプト文として記述され、このソース・スクリプト・ファイルをコンパイルすると、制御ブロック

が作成されます。作成された制御ブロックはデータベース・システムとアプリケーションが使用します。

データ・セグメントとレコードの構成および格納に使用されるデータベース・アクセス方式にはいくつかの種類があります。使用するアクセス方式の手法によってアプリケーションで使用できる機能や性能が異なるため、アプリケーションの必要条件を十分に分析してからアクセス方式を決める必要があります。

以下は、最も多く使われる階層データベース・アクセス方式です。

- Hierarchical Direct Access Method (HDAM): 階層直接アクセス方式
- Hierarchical Indexed Direct Access Method (HIDAM): 階層索引直接アクセス方式
- Hierarchical Sequential Access Method (HSAM): 階層順次アクセス方式
- Hierarchical Indexed Sequential Access Method (HISAM): 階層索引順次アクセス方式
- Data Entry Database (DEDB): 高速処理データベース

階層データベースの物理ストレージ構造のタイプは以下のとおりです。

- Hierarchical Direct

Hierarchical Direct構造は、物理的なデータベース・レコードが各セグメントのPREFIXに格納されているアドレス・ポインターによって連結されている格納構造です。HDAMとHIDAMの2つのアクセス・メソッドがあります。

- HDAMは、Hierarchical Direct構造においてルート・セグメントを検索するためにRandomizerアルゴリズムを使用します。
- HIDAMは、Hierarchical Direct構造においてルート・セグメントを検索するために索引を使用します。

- Hierarchical Sequential

Hierarchical Sequential構造は、データベース・セグメント同士の関係が物理的なデータベース・レコードの隣接で表示される格納構造です。

Hierarchical Directアクセス・メソッドの方がよりメリットが多いため、最近ではHierarchical Sequentialアクセス・メソッドはあまり使用されていません。HSAMとHISAMの2つのアクセス・メソッドがあります。

- Data Entry Database(DEDB)

DEDBは、大容量かつ高性能アプリケーション向けのアクセス・メソッドです。ただし、アプリケーションでこの機能を使用するためには、同機能を使用できるようにデータベースが設計される必要があります。

Hierarchical Direct構造とHierarchical Sequential構造のデータベースは、多くの場合、Full Functionデータベースと呼ばれます。一方、DEDBデータベースは高速パス(Fast Path)データベースと呼ばれます。IBMメインフレームではMSDB (Main Storage Database)という高速パス・データベース

がもう1つありますが、最近ではVSO(Virtual Storage Option)を使用するDEDBによって代替されています。

- Index Database

Index Databaseは、二次索引(Secondary Index)やHIDAMの主索引(Primary Index)を物理的に実装するために使用されるデータベースです。

- Generalized Sequential Access Method (GSAM)

バッチ・プログラムやBMPプログラムで、階層構造のデータベースではなく、順次データセットに対する簡単なデータベース・アクセスAPIを提供するためのアクセス・メソッドです。

1.5. サポート範囲

HiDBは、IBMメインフレームのIMS/DBに対応されるOpenFrame製品として、IMS/DBの物理構造をRDBのテーブルとして保存します。ただし例外として、GSAMは非VSAMデータセットの形式をそのまま使用します。

HiDBがサポートするアクセス方式は以下のとおりです。

- HIDAM: RDBのテーブルとしてDBDでSEQとして指定したフィールドに索引を定義します。
- HDAM: RDBテーブルとしてルート・セグメントのOCC_ID列に索引を定義します。
- GSAM: 非VSAMデータセットをそのまま使用し、HiDBの命令でアクセスできるようにサポートします。

その他のアクセス方式は、HIDAMと同様にサポートします。

HiDBでは、データベース・レコードのアクセス方式として、次の高度な機能を追加で提供しています。

- 論理関係
- 二次索引

HiDBでサポートする高度な機能の詳細については、[「第4章 高度な機能」](#)を参照してください。

第2章 データベースの管理

本章では、設計されたOpenFrame/HiDBデータベースを実装するために必要な作業について説明します。

2.1. 概要

HiDBデータベースの管理とは、データベースを設計・実装・維持することをいいます。

以下は、HiDBデータベースの各管理業務についての説明です。

- データベースの設計

導入先のアプリケーション要件を分析してデータ構造を作成します。データ構造は、新規データベースに格納されるデータの種類と、そのデータ同士の連結関係を構造的に示します。

データ構造が決定されると、データベースを設計することができます。データベースの管理者は、データベースの設計段階で定義項目とオプションなどを設定します。

- データベースの実装

データベースが設計されると、データベースの特徴およびアプリケーションがデータベースを使用する方法をマクロ・アセンブラでコーディングして、データベースを実装します。

データベースの実装は、DBD(Database Description)スクリプトとPSB(Program Specification Block)スクリプトをコーディングしてコンパイルする作業で構成されています。そのほか、ACB(Application Control Block)を定義する作業があります。

- データベースのロード

データベースが定義されると、データベースにデータをロードするために初期ロード・プログラムを作成する必要があります。データベースにデータがロードされると、アプリケーションはデータベースを使用して作業を実行することができます。メインフレームで使用していた階層データベースのデータをHiDBで使用するためには、データベースのマイグレーションを行います。

- データベースの維持

データベースの運用が開始されると、データベースの性能を継続的にモニタリングおよびチューニングする必要があります。定期的なモニタリングはデータベースのチューニング時期を決定する上で役立ちます。

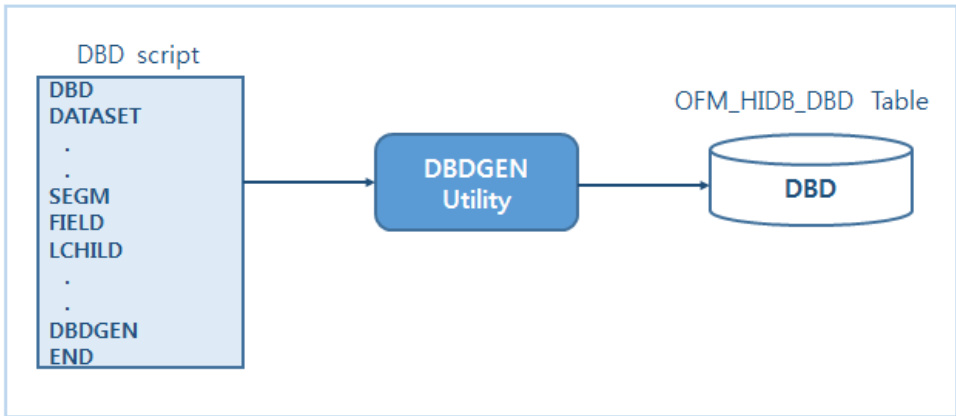
2.2. DBD

HiDBデータベースの設計が完了したら、データベースを使用するアプリケーションを実行する前に、データベース管理者は**DBD**(Database Description)のコーディングおよびコンパイルを通じて、データベースの物理的または論理的な特性をHiDBシステムに通知する必要があります。

DBDは、データベースの構造やアクセス・メソッド、データベース・レコード内のセグメントとフィールド、およびそれぞれのセグメント・タイプ間の関係について記述するマクロ・アセンブラ・コマンドの集合です。HiDBの関連文書では、DBDスクリプトに記述されているマクロ・アセンブラ・コマンドの集合を**DBDメタデータ**と呼びます。

以下は、**DBDGEN**ユーティリティ・プログラムを使用してDBD制御ブロックを作成する方法についての説明です。

[図 2.1] DBD作成プロセス



作成されたDBDマクロ・アセンブラ・スクリプトは、**DBDGEN**ユーティリティ・プログラムの入力として使用されます。このユーティリティ・プログラムはアセンブリ言語のコンパイル・プログラムで、DBDメタデータをDBD関連のメタテーブルに保存します。保存されたDBDメタデータは、以降データベースの処理時に使用されます。

参考

HiDBでは、DBD制御ブロックを作成するために、**DBDGEN**ユーティリティの代わりに**dbdgen**プログラムを提供します。

定義された各データベースごとに、1つずつの入力スクリプトが必要です。

以下は、入力スクリプトについての説明です。

区分	説明
DBD文	現在記述しているデータベース名とDL/Iアクセス・メソッドを指定します。
DATASET文	データベースの物理的なデータ・リポジトリとして使用されるデータセットの属性を指定します。データベースの種類に応じて、最大10個までのデータセット・グループを定義することができます。また、データベースに存在する各データセット・グループごとに1つのDATASET文が使用されます。

区分	説明
	各DATASET文の後にはSEGM文が続きます。SEGM文に定義されたすべてのセグメントは、該当するデータセット・グループに格納されます。
AREA文	DEDBデータベースの場合は、DATASET文の代わりにAREA文が使用されます。AREA文はDEDBのデータ領域を定義します。1つのデータベース内に複数のデータ領域を定義するために、最大240個までのAREA文が使用されます。すべてのAREA文は、DBD文と最初のSEGM文の間に位置する必要があります。
SEGM文	SEGM文は、データベースに1つのセグメント・タイプを定義するために使用されます。SEGM文には該当するセグメントの階層構造における位置、物理的な属性、他のセグメントとの関係などの情報が記述されます。セグメントは階層構造での順序どおり定義される必要があります、最大1000個、15レベルの深さまでサポートされます。 元のシステムの構文に加えて、EXTPROCとGNALLPARENTパラメータを指定できます。詳細については、「付録B データベース定義の固有の構文」を参照してください。
FIELD文	FIELD文は、1つのセグメントの特定フィールドを定義します。 FIELD文は自身が属するセグメントのSEGM文の直後に定義される必要があります。また、1つのセグメント内に存在する順次フィールドとDL/Iの呼び出し時に使用される、SSAで言及されたフィールドは、すべてFIELD文で定義します。
LCHILD文	LCHILD文は、二次索引、2つのセグメント・タイプ間の論理関係を定義します。また、HIDAM索引データベースとそのルート・セグメント間の関係を定義します。索引間の関係を定義するには、PTRパラメータにINDX、SYMBを指定し、論理関係を定義するには、PTRパラメータにSNGL、DBLE、NONEを指定します。
XDFLD文	XDFLD文は、ソース・セグメントとターゲット・セグメントを関連付けます。二次索引が使用される場合にのみ使用します。XDFLD文には、索引フィールド名、ソース・セグメント名、二次索引キーの作成に使用されるフィールド名を指定します。 – SRCH: ユーザー・アプリケーションで検索条件として指定するフィールドを指定します。 – SUBSEQ: 検索キーとしては使用されませんが、SRCHフィールド以外にHIDBが追加で二次索引キーとして管理するフィールドを指定します。 – DDATA: 検索キーとして使用されずに、索引セグメントの追加データとして使用するフィールドを指定します。

区分	説明
DBDGEN文, END文	DBDGEN文とEND文はDBDの作成スクリプトの最後に追加され、それぞれ以下のような意味を持ちます。 – DBDGEN : DBDを定義する最後の構文 – END : 入カスクリプトの最後の構文

2.3. PSB

特定のアプリケーションはデータベースに格納されているすべてのセグメントとフィールドを使用する必要はありません。また、セキュリティや統合管理上の理由で特定のセグメントやフィールドへのアクセスを防止しなければならない場合があります。

データベース管理者は、**PSB**(Program Specification Block)をコーディング、コンパイルして、特定のアプリケーションがデータベースから読み込むセグメントとフィールド、およびセグメントに対して行える作業を制御できます。PSBはデータベース・セグメントに対するアプリケーションのアクセス方式を記述するマクロ・アセンブラ・コマンドの集合です。1つのPSBは1つ以上のPCB(Program Communication Block)で構成されており、各PCBは関連するデータベースの読み込みや使用できるアプリケーションの権限を記述します。HiDBの関連文書では、PSBスクリプトに記述されているマクロ・アセンブラ・コマンドの集合を**PSBメタデータ**と呼びます。

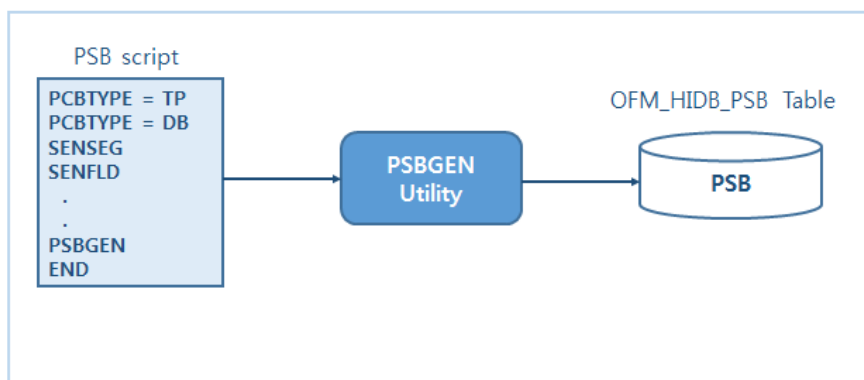
アプリケーションは使用するPSBに複数のPCBを持つことにより、同じデータベースに対する複数のビューを持つことができます。また、複数のデータベースに同時にアクセスすることもできます。

参考

データベースに対するアプリケーションの権限について記述するデータベースPCBのほかに、「代替PCB」と呼ばれる別のタイプのPCBがあります。これは、オンライン・プログラムで「Alternate Message Destination」を記述する際に使用されます。

以下は、PSBGENユーティリティ・プログラムを使用してPSBメタデータをメタテーブルに保存する方法についての説明です。

[図 2.2] PSB作成プロセス



作成されたPSBマクロ・アセンブラ・スクリプトは、**PSBGEN**ユーティリティ・プログラムの入力として使用されます。このユーティリティ・プログラムはアセンブリ言語のコンパイル・プログラムで、PSBメタデータをPSB関連のメタテーブルに保存します。保存されたPSBメタデータは、以降データベースの処理時に使用されます。

参考

HiDBでは、PSB制御ブロックを作成するために、PSBGENユーティリティの代わりに**psbgen**ツールを提供します。

以下は、入カスクリプトについての説明です。

区分	説明
代替PCB文	2種類の異なるPCB文を入カスクリプトに作成できます。 そのうち、1つ目のPCBタイプである代替PCBは、オンライン・トランザクション・メッセージが最初に作成された場所とは異なる場所に転送するためのデスティネーションを作成します。 代替PCB文は入カスクリプトの先頭に位置する必要があります。
データベースPCB文	もう1つのPCBタイプであるデータベースPCB文は、アプリケーションがアクセスするデータベースのDBD名を指定します。また、PCB文ではアプリケーションがデータベースに格納されているセグメントに対して実行できる作業(GET、INSERT、REPLACEなど)を定義します。 この構文で記述されるデータベースは物理データベースあるいは論理データベースになります。アプリケーションがアクセスするすべてのデータベースに対し、独立したデータベースPCBが必要です。
SENSEG文	SENSEG文は、アプリケーションで使えるデータベースのセグメント・タイプを定義します。それぞれのセグメントに対して独立的なSENSEG文が必要です。
SENFLD文	SENFLD文は、Field-Level Sensitivity機能と併用される必要があります。また、アプリケーションで使えるセグメントのフィールドを定義します。
PSBGEN文	PSBGEN文はPSBの名前を指定し、アプリケーションの各種属性を定義します。アプリケーションで使われたプログラム言語やI/Oバッファの最大サイズなどの属性が定義されます。入カスクリプトには1つのPSBGEN文が必要です。
END文	PSB作成スクリプトの最後にはEND文が定義されます。END文は、psbgenツールに入カスクリプトの終わりを通知します。

2.4. ACB

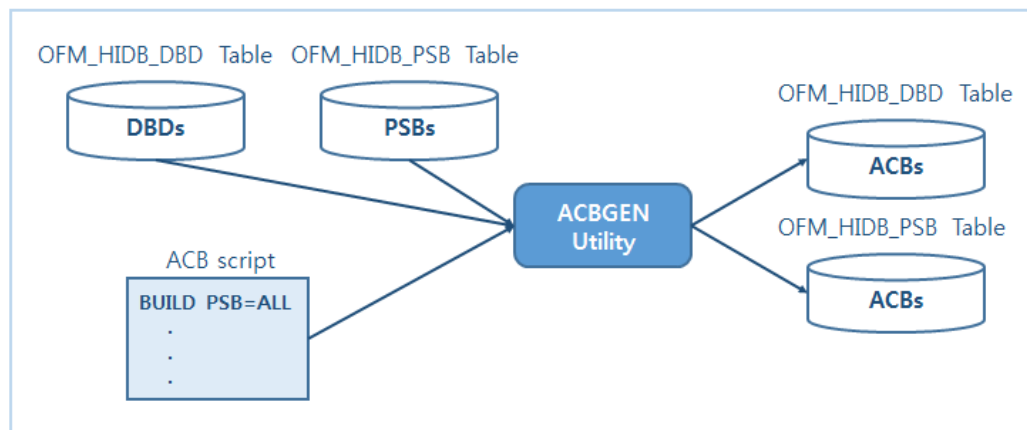
ACBGENユーティリティ・プログラムは、PSBメタデータとDBDメタデータをまとめて、ACB(Application Control Block)メタデータを作成します。オンライン・アプリケーションでHiDBデータベースを使用するためには、オンライン・サーバーを起動する前に、必ずACBメタデータを作成する必要があります。

参考

HiDBでは、ACB制御ブロックを作成するために、ACBGENユーティリティの代わりに**acbgen**ツールを提供します。

以下は、ACBGENユーティリティ・プログラムを使用してACBメタデータをACBメタテーブルに保存する方法についての説明です。

[図 2.3] ACB作成プロセス



ACB管理ユーティリティのACBGENプログラムは、ACBメタデータに必要なPSBメタデータとDBDメタデータをPSB関連のメタテーブルとDBD関連のメタテーブルから取得します。

ACBGENユーティリティ・プログラムを使用すると、PSB関連のメタテーブルにあるすべてのPSBに対してACBメタデータを作成できます。また、特定のPSB名またはDBD名を指定して、該当するPSBに対するACBメタデータを作成することもできます。作成されたACBメタデータはPSB関連のメタテーブルとDBD関連のメタテーブルに保存されます。

特定のオンライン・プログラムの実行スケジュールが作成されると、オンライン・プログラムで使用するPSBに該当するACBメタデータがすぐにメモリーにロードされます。そのため、特定のオンライン・プログラムの実行をスケジュールする時間を短縮できます。

2.5. 可変長セグメント

可変長セグメントを定義するためには、DBDのSEGMENT句BYTES項目を(max,min)形式で記述します。可変長セグメントを有するデータベースにアクセスするアプリケーション・プログラムでは、

DL/I関数を呼び出すとき、セグメントI/O領域の最初2バイトに実際に処理するセグメント長の情報を記述する必要があります。

HiDBでは、可変長セグメントを一般セグメントと同じ方法で処理しますが、セグメントを定義する際に使用するコピーブックをdbdcpybkgenで作成しなければならないという制約があります。

第3章 DL/I関数の呼び出し

本章では、DL/I関数の使用方法について説明します。

3.1. DL/I関数

DL/I(Data Language/I)は、COBOLまたはPL/I言語で作成されたアプリケーションでHiDBにアクセスするために呼び出すインターフェース・モジュールです。つまり、一般のアプリケーションがHiDBのデータを検索または更新するためには、DL/Iが提供する機能と形式に従って呼び出す必要があります。

次は、COBOLで使用するDL/I呼び出し文の構文です。

```
CALL  'CBLTDLI'  USING  DLI-function
                        PCB-mask
                        Segment-io-area
                        Segment-search-arguments
```

CBLTDLIは、COBOLでDL/Iを呼び出す際に使用されるインターフェース・モジュール名です。

次は、COBOLでAIBを使用して呼び出すDL/I呼び出し文の構文です。

```
CALL  'AERTDLI'  USING  DLI-function
                        AIB-mask
                        Segment-io-area
                        Segment-search-arguments
```

AERTDLIは、COBOLでAIBを使用してDL/Iを呼び出す際に使用するインターフェース・モジュール名です。

次は、PL/Iで使用するDL/I呼び出し文の構文です。

```
CALL  'PLITDLI' (      Parameter count
                        DLI-function
                        PCB-mask
                        Segment-io-area
                        Segment-search-arguments      )
```

PLITDLIは、PL/IでDL/Iを呼び出す際に使用されるインターフェース・モジュール名です。

DL/I関数の呼び出し時に渡すパラメータは事前に適切な値を設定します。DL/I呼び出しは、呼び出し文と呼び出しに必要な各種の情報を含んでいるパラメータで構成されます。

以下は、DL/I呼び出しに使用されるパラメータです。Parameter countは、PLITDLIモジュールを使用する場合にのみ記述します。

パラメータ	説明
Parameter count	PLITDLIモジュールでのみ使用するパラメータです。このパラメータ以外のパラメータの数を記述します。
DLI-function	セグメントの検索、挿入、削除、更新などDL/Iが提供する様々な機能の中で、アプリケーションで使いたい機能を定義します。
PCB-mask	アプリケーションでDL/Iを呼び出した後に返されるステータス・コード、セグメント名などを格納するために定義されたデータ構造体パラメータです。
Segment-io-area	セグメントの検索後に返されるデータを格納するか、セグメントの挿入・更新のためのデータをDL/Iに渡すために使用します。
Segment-search-arguments	検索または処理したい特定のセグメントの検索条件を記述します。1つ以上のセグメント名と条件文で構成されます。
AIB-mask	アプリケーションでAERTDLIインターフェースを使用してDL/Iを呼び出した後に返されるリターンコードとPCBマスク・アドレスを保存するために定義されたデータ構造体パラメータです。

3.2. DL/I機能

DL/I呼び出し文の最初のパラメータを使ってDL/Iで提供する各種機能の中から使いたい機能を設定します。DL/Iが提供する機能は、大きく3つに分けることができます。

- Get

データベースからセグメントのデータを検索するために使用する機能です。

機能	説明
GU(Get Unique) *GHU(Get Hold Unique)	特定のセグメントを直接検索します。順次処理する最初のセグメント・インスタンスの位置を指定する際に使用します。検索する対象の条件は、SSAパラメータに記述します。
GN(Get Next) *GHN(Get Hold Next)	セグメントを順次検索します。現在の位置から階層パスに従って次のセグメント・インスタンスに移動して検索します。
GNP(Get Next in Parent) *GHNP(Get Hold Next in Parent)	GNと同様な機能であり、親セグメントに従属するセグメントのみを順次検索します。親セグメントはGU機能によって検索されたセグメントで事前に設定されます。

- Update

データベースにデータを挿入、修正、削除するために使用する機能です。

機能	説明
ISRT (Insert)	セグメントに新規データを追加する際に使用します。
DLET (Delete)	セグメントのデータを削除する際に使用します。
REPL (Replace)	セグメントのデータを変更する際に使用します。

- その他

システム・サービスに関連する機能です。

機能	説明
CHKP, SYMCHKP(Checkpoint)	RDBのコミットと類似しており、プログラムによって変更されたすべての内容をデータベースに適用します。SYMCHKP(シンボリック・チェックポイント)で渡されたアドレスのデータをバックアップします。
XRST(Extended Restart)	SYMCHKPでバックアップしたデータを元のアドレスに戻します。
SYNC(Synchronization Checkpoint)	CHKPと同じ機能を実行します。
ROLL, ROLB(Rollback)	最後のコミット以降のすべてのデータ変更をロールバックします。ROLLの場合、終了コードのU007788を使用してプログラムを終了します。
APSB(Allocate PSB)	使用するリソース(PSB)をロードしてメモリを割り当てます。AERTDLIインターフェースを使用してDLI関数を呼び出す前に、この機能を呼び出す必要があります。
DPSB(Deallocate PSB)	APSBを使用して割り当てたリソースを解除します。AERTDLIインターフェースの使用を終了するときに呼び出されます。

参考

*H(Hold)文字が付いているGet機能はセグメントを修正(REPL)、削除(DLET)する前に使用される機能です。REPLとDLET機能を呼び出す前に必ずH文字が付いているGet機能を使用してセグメントを検索する理由は、他のユーザーによって当該データが修正、削除されないようにロックをかけるためです。

3.3. PCBマスク

IMSシステムでは、アプリケーションのDLI関数の呼び出しの結果情報をPCBに設定します。アプリケーションは、PCB構造を含んでいるPCBマスクとPCBマスクの各フィールドを参照して、DLI呼び出しへの成功または失敗を確認することができます。

DL/I呼び出しの2番目のパラメータであるPCBマスクは、Database name、Segment level number、Status Codeなどの9つの項目で構成されており、開発者はその中から5つの項目を参照します。特にStatus Codeは、DL/Iの呼び出しに関する処理結果コードが格納される項目であり、アプリケーションで処理結果に基づいて作業を分岐するときに比較対象として頻繁に使用されます。

segment level numberまたはsegment name項目は、DL/Iが最終的に作業したセグメントの情報が格納されるので、非修飾GNまたはGNPの呼び出しで最後に検索したセグメントを確認することができます。また、エラーが発生した場合も、この情報を参照してエラーの対象セグメントを把握できます。

以下は、COBOLプログラムで実装したPCBマスクの例です。

```
01  SEGMENT1-PCB-MASK.
    05  SPCB-DBD-NAME          PIC  X(8).
    05  SPCB-SEGMENT-LEVEL     PIC  X(2).
    05  SPCB-STATUS-CODE       PIC  X(2).
    05  SPCB-PROC-OPTIOINS     PIC  X(4).
    05  FILLER                 PIC  S9(5)  COMP.
    05  SPCB-SEGMENT-NAME      PIC  X(8).
    05  SPCB-KEY-LENGTH        PIC  S9(5)  COMP.
    05  SPCB-NUMB-SENS-SEGS    PIC  S9(5)  COMP.
    05  SPCB-KEY               PIC  X(11).
```

以下は、PCBマスクの各フィールドについての説明です。

フィールド名	長さ(Byte)	説明
Database name	8	アクセスしたセグメントが属しているデータベース名が格納されます。
Segment level number	2	データベース内の現在のセグメント・レベル数です。アクセスしたセグメントがルート・セグメントの場合、01が格納されます。
Status Code	2	DL/Iの呼び出し後に関数の処理結果に対するステータス・コードが設定されます。詳細については、 「付録A. DL/Iステータス・コード」 を参照してください。
Processing options	4	プログラムで使用可能な呼び出しタイプのコードが設定されます。プログラムで実行可能な処理を示すコードであり、PSBGENを実行する際にPROCOPTパラメータに指定した値が渡されます。
Reserved for IMS	4	IMSが内部的に使用するフィールドであり、アプリケーションでは使用しません。
Segment name	8	正常に呼び出しが終了した場合、アクセスしたセグメントの中から最後のセグメント名が設定されます。呼び出しに失敗した場合は、要求したセグメントの中で最後のセグメント名が格納されます。

フィールド名	長さ(Byte)	説明
Length of key	4	Key feedback areaの現在の長さが格納されます。
Number of sensitive segments	4	バイナリ・フィールドであり、アプリケーションが検知できるセグメント・タイプの番号が格納されます。
Key feedback area	var length	検索または挿入時にアクセスしたセグメントの連結キー (Concatenated key)が格納されます。連結キーとは、アクセスした全レベルのセグメント・キー・フィールド値を順次に組み合わせたキーです。

ステータス・コード

ステータス・コードは、DL/I呼び出しの処理結果の情報を示すコードです。アプリケーションでこの情報を参照して適切な処理を行います。DL/I呼び出しが正常に完了すると、空白の値を返します。

以下は、主なステータス・コードについての説明です。

コード	説明
GA	最下位レベルのセグメント・レコードを読み込んだ後、次の上位セグメントを読み込む前に返されるコードです。修飾呼び出しでは発生しません。最下位の階層セグメントのインスタンスがそれ以上存在しない場合に発生しますが、処理には影響を与えない情報性コードです。
GB	順次検索される際、階層パスに従ってすべてのデータを検索し、データベースの最後に達したときに返されるコードです。データベースの最後を示すコードであり、GNでは発生しますが、GNPでは発生しません。
GE	提示した条件に一致するセグメント・インスタンスを検索できない場合に返されるコードです。
GK	非修飾GNあるいはGNPを呼び出すとき、同じ階層レベルで異なるセグメント・タイプが見つかった場合に返されるコードです。処理には影響を与えない情報性コードです。

参考

GAとGKは、処理が正常に完了した場合に返される情報性ステータス・コードです。これらは頻繁に返されるコードなので、その概念を熟知しておく必要があります。

以下は、各関数ごとに返される主なステータス・コードです。各ステータス・コードの詳細については、「[付録 A. DL/Iステータス・コード](#)」を参照してください。

	GU/GHU	GN/GHN	GNP/GHNP	REPL/DLET	ISRT
GA	-	○	○	-	-

	GU/GHU	GN/GHN	GNP/GHNP	REPL/DLET	ISRT
GB	—	○	—	—	—
GE	○	○	○	—	—
GK	—	○	○	—	—

3.4. AIBマスク

AERTDLIインターフェースでは、AIBマスクを使用してDL/I関数を呼び出します。

アプリケーションは、IMSシステムにアクセスするために使用するリソース名をAIBマスクに設定します。AERTDLIでは、PCBマスクのアドレスがAIBマスクに渡され、アプリケーションによって参照されます。

以下は、COBOLプログラムに実装されたAIBマスクの例です。

```
01 AIB.
  05 AIBRID PIC x(8).
  05 AIBRLen PIC 9(9) USAGE BINARY.
  05 AIBRSFUNC PIC x(8).
  05 AIBRSNM1 PIC x(8).
  05 AIBRSNM2 PIC x(8).
  05 AIBRESV1 PIC x(8).
  05 AIBOALEN PIC 9(9) USAGE BINARY.
  05 AIBOAUSE PIC 9(9) USAGE BINARY.
  05 AIBRESV2 PIC x(12).
  05 AIBRETRN PIC 9(9) USAGE BINARY.
  05 AIBREASN PIC 9(9) USAGE BINARY.
  05 AIBERRXT PIC 9(9) USAGE BINARY.
  05 AIBRESA1 USAGE POINTER.
  05 AIBRESA2 USAGE POINTER.
  05 AIBRESA3 USAGE POINTER.
  05 AIBRESV4 PIC x(40).
  05 AIBRSAVE OCCURS 18 TIMES USAGE POINTER.
  05 AIBRTOKN OCCURS 6 TIMES USAGE POINTER.
  05 AIBRTOKC PIC x(16).
  05 AIBRTOKV PIC x(16).
  05 AIBRTOKA OCCURS 2 TIMES PIC 9(9) USAGE BINARY.
```

次は、AIBマスクの各フィールドについての説明です。ただし、HiDB AERTDLIで使用されるフィールドについてのみ説明しています。

フィールド名	長さ(Byte)	説明
Resource name	8	APSB、DPSBを呼び出す場合は、PSB名を指定する必要があります。他のDL/Iを呼び出す場合には、PCB名を指定します。

フィールド名	長さ(Byte)	説明
		(例: AIBRSNM1、AIBRSNM2)
Resource address		DL/I関数を呼び出した後、PCBマスクのアドレスが返されます。 (例: AIBRESA1、AIBRESA2..)
Return code	4	AIBTDLIのリターンコードが返されます。成功の場合は0が返され、その他の場合はエラーコードが返されます。 (例 : AIBRETRN)

3.5. SSA

SSA(Segment Search Argument)は、DL/I呼び出しに対する処理プロセスに必要な条件情報が含まれているパラメータです。RDBのWHERE句と同様な機能です。DL/I呼び出し時にSSA内に1つ以上の条件文が存在するかどうかによって、非修飾呼び出しと修飾呼び出しに分類されます。

非修飾SSA

アクセス対象のセグメント名のみを指定する場合に使用します。セグメント名の最大長は8バイトであり、末尾に1つの空白を追加します。セグメント名が8バイトより小さい場合は、残りの部分を空白で埋めます。

その形式は以下のとおりです。

[図 3.1] 非修飾SSAの形式



以下は、COBOLプログラムで実装した非修飾SSAの例です。

```

01  UNQUALIFIED-SSA.
   05  UNQUAL-SSA-SEGMENT-NAME  PIC  X(08).
   05  FILLER                    PIC  X   VALUE  SPACE.
```

修飾SSA

特定の値を有するセグメントにアクセスするには、フィールドの条件文を記述した修飾SSAを使用します。その形式は以下のとおりです。

[図 3.2] 修飾SSAの形式



- セグメント名
アクセスするセグメント名を指定します。
- フィールド名
条件の比較対象となるセグメントのフィールド名を指定します。フィールド名はDBDスクリプトのFIELDに記述されているフィールド名と一致する必要があります。
- 関係演算子
指定したフィールドと検索値との関係を示す演算子を指定します。演算子の長さは2バイトであり、指定できる演算子は以下のとおりです。

関係	演算子
一致(Equal to)	EQ
	=□
	□=
不一致(Not equal to)	NE
	^=
	=^
超過(Greater than)	GT
	>□
	□>
以上(Greater than or equal to)	GE
	>=
	=>
未満(Less than)	LT
	<□
	□<
以下(Less than or equal to)	LE
	<=

関係	演算子
	=<

参考

□は1Byteの空白を意味します。

● 検索値

フィールド名に対応する値であり、指定値を基準にして関係演算処理を行います。

以下は、COBOLプログラムで実装した修飾SSAの例です。

```
01  SEGMENT-SSA.
    05  FILLER          PIC  X(9)    VALUE  'SEGMENT1(' .
    05  FILLER          PIC  X(10)   VALUE  'SEG1CODE =' .
    05  SEG1-SSA-CODE   PIC  X(3) .
    05  FILLER          PIC  X       VALUE  ' )' .
```

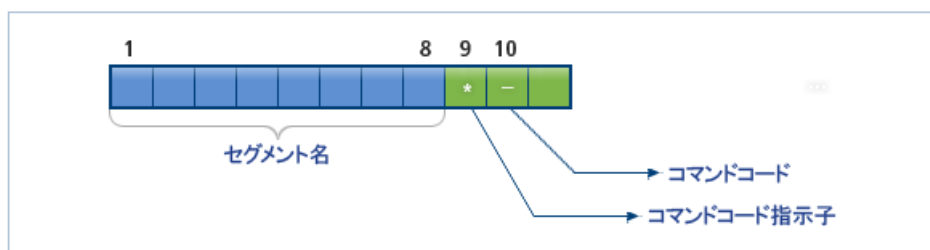
コマンド・コード

SSA内の指定された位置にコマンド・コードを設定すると、DL/I関数の様々な機能を拡張して使用できます。コマンド・コードは1つの文字で表し、SSA内のセグメント名の次の位置である9バイトの位置にコマンド・コードの指示子であるアスタリスク(*)を設定してから、10バイトの位置に使用したいコマンド・コードを設定します。

コマンド・コードは非修飾SSAと修飾SSAの両方に使用可能であり、使用形式は以下のとおりです。

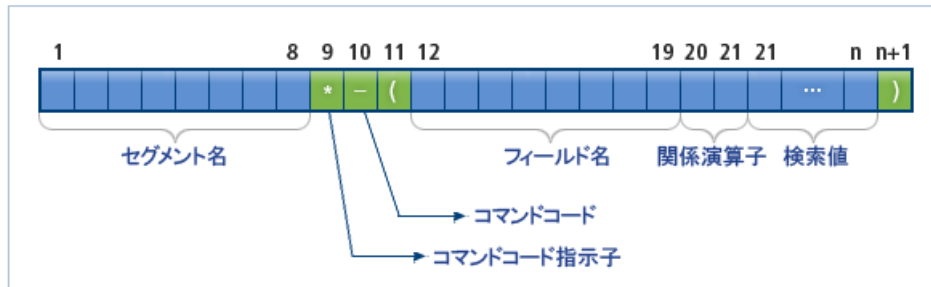
● 非修飾SSAの形式

[図 3.3] 単一コマンド・コードを持つ非修飾SSA形式



- 修飾SSAの形式

[図 3.4] 単一コマンド・コードを持つ修飾SSA形式



以下は、COBOLプログラムでコマンド・コードを使用して非修飾SSAを実装した例です。

```

01 UNQUALIFIED-SSA.
   05 UNQUAL-SSA-SEGMENT-NAME PIC X(08).
   05 FILLER PIC X VALUE " * ".
   05 UNQUAL-SSA-COMMAND-CODE PIC X VALUE " - ".
   05 FILLER PIC X VALUE SPACE.
  
```

以下は、使用可能なコマンド・コードの一覧です。

コード	説明
C	複数セグメントのキー・フィールド値を連続して指定した連結キーを使用して、データを検索または挿入します。
D	Path callとも呼ばれており、階層パスに従って指定したセグメントに順次アクセスし、アクセスしたすべてのセグメントのデータをI/O領域に順番に返します。
F	セグメントの最初の位置を意味します。検索や挿入の処理時にセグメントの最初の位置で指定した作業を行います。つまり、Get Nextは、Get Uniqueのように動作します。
L	セグメントの最後の位置を意味します。検索や挿入の処理時にセグメントの最後で指定した作業を行います。
*M	サブセット・ポインターを次のセグメント・インスタンスに移動させます。
N	REPLの呼び出し時に使用されるコマンド・コードです。Dコマンド・コードによってpath callで検索されたデータの更新を防止するために使用されます。
P	特定のセグメントに親セグメントを設定して、GNPの呼び出し時に使用できるようにします。一般的にアクセスしたセグメントの中で最下位レベルのセグメントが親セグメントとして設定されますが、Pコマンド・コードを使ってそれより上位レベルのセグメントを親セグメントとして設定できます。Pコマンド・コードを使用したGET要求が失敗した場合にも、Pコマンド・コードが指定されたセグメント・レベルまでの検索に成功したら、そのレベルを親セグメントとして設定します。

コード	説明
Q	検索したセグメントの内容が別のプログラムで更新されることを防ぐ目的で使用されます。
*R	最初のサブセットが指すセグメントのデータを検索します。
*S	現在の位置を無条件にサブセット・ポインターに設定します。
U	現在のセグメントの位置を維持させます。セグメントを検索した場合、目的のデータが検索されるか、それ以上のデータが存在しなくなるまで、現在のセグメントの位置が階層構造に従って移動しますが、Uコマンド・コードを指定したセグメントは現在の位置を維持します。
V	Uコマンド・コードと基本的には同様な機能ですが、Vコマンド・コードを指定したセグメントの上位レベルのセグメントの位置も維持させます。
*W	サブセット・ポインターが未設定の場合、現在の位置に設定します。
*Z	サブセット・ポインターを0に初期化して再使用できるようにします。
—	Nullコマンド・コードです。コマンド・コードをハイフン(-)で設定する場合、非修飾SSAとして処理します。プログラムの途中で、コマンド・コードを再設定して必要な作業を処理することもできます。

参考

アスタリスク(*)が付いているコードはDEDBでのみ使用されます。

以下は、コマンド・コード別に使用可能なDL/I関数です。

	GU/GHU	GN/GHN	GNP/GHNP	REPL	ISRT	DLET
C	○	○	○	-	○	-
D	○	○	○	-	○	-
F	○	○	○	-	○	-
L	○	○	○	○	○	-
M	○	○	○	○	○	-
N	-	-	-	○	-	-
P	○	○	○	-	○	-
Q	○	○	○	-	○	-
R	○	-	-	-	-	-
S	○	○	○	○	○	-
U	○	○	○	-	○	-

	GU/GHU	GN/GHN	GNP/GHNP	REPL	ISRT	DLET
V	○	○	○	—	○	—
W	○	○	○	○	○	—
Z	○	○	○	○	○	○
—	○	○	○	○	○	○

多重条件文

SSA内でブール演算子を各条件文の間に定義し、複数の条件文を結合して使用することができます。

関係	演算子
AND(2つ以上の条件を満たす場合)	「&」 または 「*」
OR (1つ以上の条件を満たす場合)	「 」 または 「+」

以下は、COBOLプログラムで多重条件文を使用してSSAを実装した例です。

```

01  SEGMENT1-SSA.
    05  FILLER          PIC  X(9)    VALUE  'SEGMENT1(' .
    05  FILLER          PIC  X(10)   VALUE  'SEG1CODE>=' .
    05  SEG1-SSA-LOW-CODE PIC  X(3) .
    05  FILLER          PIC  X       VALUE  '&' .
    05  FILLER          PIC  X(10)   VALUE  'SEG1CODE<=' .
    05  SEG1-SSA-HIGH-CODE PIC X(3) .
    05  FILLER          PIC  X       VALUE  ')' .

```

第4章 高度な機能

ユーザー・アプリケーションの処理に最適なOpenFrame/HiDBデータベースを選択したら、次にOpenFrame/HiDBの高度な機能を使用するかどうかを選択する必要があります。

本章ではOpenFrame/HiDBの高度な機能である論理関係(Logical Relationship)と二次索引(Secondary Indices)について説明します。

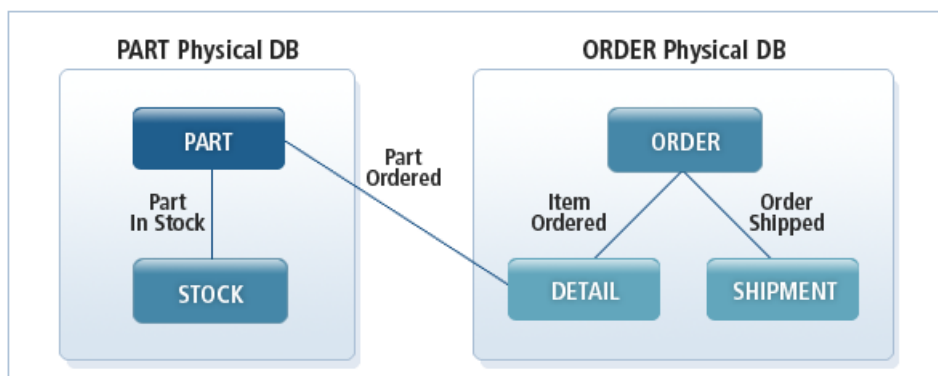
4.1. 論理関係(Logical Relationship)

HiDBシステムでは、論理関係を通じて物理的に異なる2つのデータベースを結合して使用できる機能を提供します。それにより、論理関係に含まれている特定のセグメントに対して、付加的なアクセス方式を提供する新しい階層構造が定義されます。

データベース管理者は、論理関係に参加する物理データベースを使用して、論理データベースと呼ばれる新しいデータベースを定義します。論理データベースは、前述のようにアプリケーションに新しい階層構造の形式を提供します。

以下は、アプリケーションでよく使われるデータ・アクセス方式に従って、データベースを設計段階からPARTデータベースとORDERデータベースの2つの物理データベースで実装した例です。

[図 4.1] 2つの物理データベース間の関係

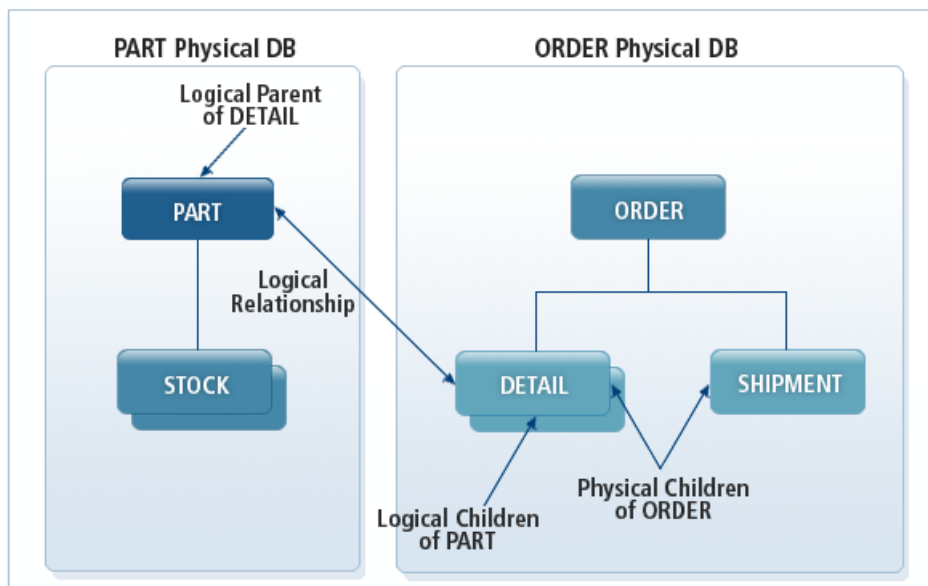


しかし、特定のアプリケーションでは、PARTデータベースのPARTセグメントとORDERデータベースのDETAILセグメント間の関係を使用する必要があるため、PARTデータベースとORDERデータベースの論理関係を定義します。

論理関係を定義する基本的な方法は、両方のデータベースと関連のある特定のセグメントを別のデータベースに存在する論理親セグメントと関連付けて、論理子セグメントとして指定する方法です。このように指定すると、論理子セグメントは物理親セグメントと論理親セグメントの2つの親セグメントを持つことになります。

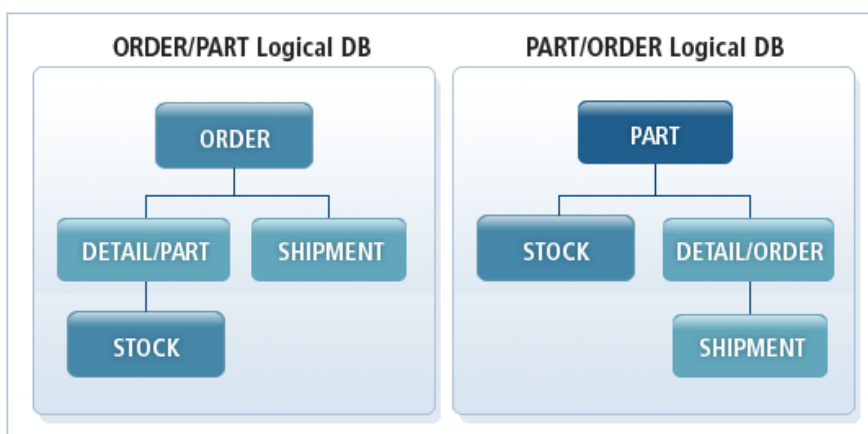
以下の図に示したように、論理子セグメントであるDETAILのインスタンスが1つ追加されると、2つの階層構造に参加することになります。DETAILセグメント・インスタンスは、物理親セグメントであるORDERと論理親セグメントであるPARTを同時に持ちます。

[図 4.2] 論理関係にある2つの物理データベース



付加的な2つの論理データベースを定義すると、以下の図のように新しい2つの論理データ構造が得られます。この新しいデータ構造は、1つのアプリケーションで同時に使用できます。

[図 4.3] 2つの付加的な論理データベース



上記図のDETAIL/PARTは、論理子セグメントと論理親セグメントを連結して作成されたセグメントであり、DETAIL/ORDERは、論理子セグメントと物理親セグメントを連結して作成されたセグメントです。これらのセグメントを**連結セグメント**(Concatenated Segment)といいます。

同じ論理親セグメント・インスタンスを持つ論理子セグメント・インスタンスを**論理ツイン・セグメント**と呼びます。論理子セグメントにアクセスする方法は、物理親セグメントを通じてアクセスする方法と、論理親セグメントを通じてアクセスする方法の2つがあります。

OpenFrame/HiDBシステムで論理関係を使用する理由は以下のとおりです。

- 論理関係は、特定のセグメントにアクセスする基本的なアクセス・パスとは異なるアクセス・パスをアプリケーションに提供します。たとえば、1つの物理データベースの特定のセグメントから別の物理データベースの関連セグメントに直接アクセスする方式を提供します。
- 論理関係は、代替階層データ構造をアプリケーションに提供します。これにより、アプリケーションは物理データベースのデータを目的に合わせて処理できます。
- 論理関係は、OpenFrame/HiDBが物理的に分離された2つのデータベースに関連付けられているセグメントの関係を強制的に維持できるようにします。たとえば、論理子セグメントがある場合は、論理親セグメントを削除できないように設定し、論理親セグメントがない場合は、論理子セグメントを挿入できないように設定できます。

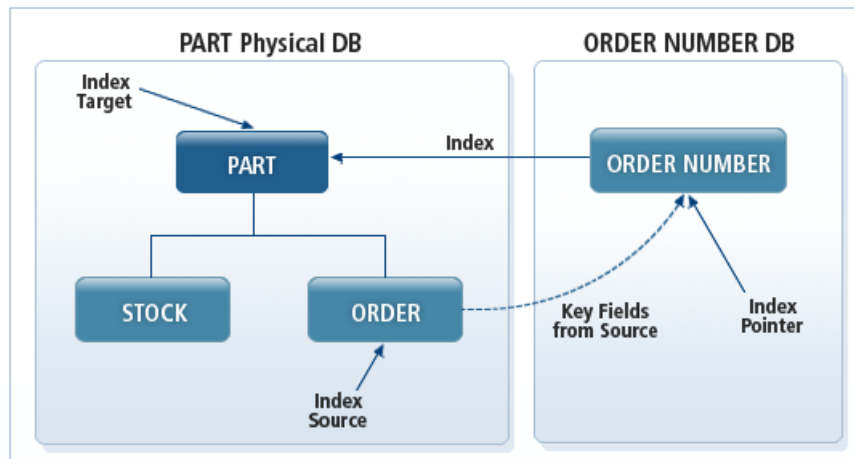
しかし、論理関係を使用すると、論理関係に使用されるポインターの管理のため性能低下が発生する可能性があります。また、データベースを再編成する場合は、論理的に連結されている他のデータベースも一緒に再編成される必要があります。したがって、論理関係の使用を決定する前に、論理関係の使用によって発生する可能性のある性能上の問題や管理業務の増加などについて十分に考慮しなければなりません。

4.2. 二次索引(Secondary Indices)

HiDBシステムは、二次索引データベースを使用して特定のセグメントにアクセスする付加的な方法を提供します。各二次索引は、ルート・セグメントのキーを使用した方式以外のアクセス方式を提供します。このようなアクセス方式は、既存の方法に比べてより迅速なデータの取得を実現します。

以下の図のように、PARTセグメントとORDERセグメントはORDERセグメントに格納されているORDER NUMBERを基準にして迅速なアクセスが可能です。

[図 4.4] 二次索引データベースを持つデータベース



二次索引は、OpenFrame/HiDBシステムに二次索引データベースを定義することによって実装されます。このデータベースは二次索引のキーを主キーとし、主データベースに存在する特定のセグメントを指すポインターを格納します。

以下は、二次索引の実装に使用されるセグメントです。

用語	説明
Index Source	ORDER NUMBERなどの索引キーがデータ・フィールドとして格納されている主データベース・セグメントを意味します。
Index Pointer	主データベースのIndex Targetセグメントを指す索引データベースのセグメントを意味します。なお、Index Sourceセグメントから取得したフィールド・データをキーとして格納します。
Index Target	二次索引機能によって初めてアクセスされる主データベースのセグメントを意味します。 二次索引のキーは、1つから5つまでのIndex Sourceセグメントのフィールドで構成されます。このように組み合わせた検索フィールド値は、オーバーヘッドを減らすために、なるべく一意の値で定義することをお勧めします。

以下は、検索フィールドの末尾に付けられ、検索フィールドが一意の値を持つようにするシステム・フィールドです。

フィールド	説明
/SX変数	Index Sourceセグメントを一意として定義するシステム変数です。
/CX変数	Index Sourceセグメントの連結キー(Concatenated Key)です。

以下は、HiDBシステムで二次索引を使用する理由です。

- データベースが定義されるとき、指定された主キー以外のキーを使用してデータベース・レコードに迅速にアクセスできます。
- Index Targetセグメントにアクセスするとき、データベースのすべての階層を走査せず、すぐにアクセスできます。
- 検索フィールドの情報だけで処理するバッチ・プログラムで、索引データベースを別途に使用できるというメリットがあります。

ただし、二次索引を使用すると、主データベースのIndex Sourceセグメントが変更されるたびに、索引データベースも同時に更新しなければなりません。また、二次索引データベースを主データベースと一緒にモニタリング、バックアップ、チューニングする管理業務が増えます。

論理関係と同様、二次索引を使用する場合は、二次索引の特長、性能上の問題、管理業務の増加などを十分に考慮してから使用してください。

二次索引のセグメント・フィールドに重複するキー値が存在する場合は、索引DBDでオーバーフロー・データセットを指定して処理するようにします。重複するキーの保存および検索は、LIFO(後入先出法)の順で処理されます。

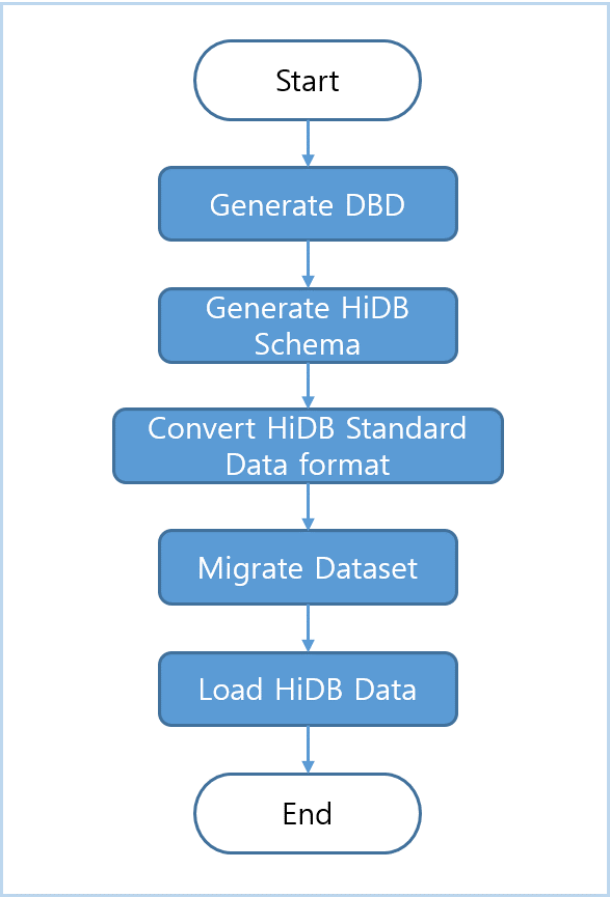
第5章 マイグレーション

本章では、メインフレーム環境で運用していたIMS/DBのデータをOpenFrame環境のOpenFrame/HiDBで運用するためのデータ・マイグレーションの手順と、各手順の実際の作業方法および注意事項について説明します。

5.1. マイグレーションの手順

以下の図は、OpenFrame環境での一般的なHiDBマイグレーションの手順を示しています。以下では、マイグレーションの手順について、DBD、PSB関連のメタテーブルが事前に作成されていると仮定して説明します。

[図 5.1] OpenFrame/HiDBマイグレーションの手順



マイグレーションの各フェーズで使用するツールは以下のとおりです。

	ツール	説明
1	dbdgen	DBD作成ツール

	ツール	説明
2	hdgensch	HiDBのスキーマ・ファイル作成ツール
3	hdpcdf01	HiDBの事前変換ツール
4	dsmigin	データセット・インポート移行ツール
5	hdload	HiDBのデータ・ロード・ツール

参考

HiDBマイグレーションの各フェーズの詳細については、各節を参照してください。

5.2. DBDの作成

HiDBマイグレーションの最初の段階では、マイグレーション対象データベースのDBD(Database Description Block)を作成します。DBDは、UNIXコマンドラインで**dbdgen**ツール・プログラムを使用して作成できます。

この際、メインフレームで使したDBD作成用のスクリプトまたはJCLが必要です。DBDを作成する前に、ims.confで指定したデフォルトDBDLIB、あるいはユーザーが定義したDBDLIBのPDSが既に作成されている必要があります。

この際、メインフレームで使したDBD作成用のスクリプトまたはJCLが必要です。DBDを作成する前に、OpenFrame環境設定の**ims**サブジェクトで指定されたデフォルトのDBDLIBまたはユーザー定義のDBDLIBのPDSデータセットが事前に作成されている必要があります。

以下は、UNIXコマンドラインでdbdgenツール・プログラムを使用して実行する例です。

```
$ dbdgen TMAXDB01
```

上記のコマンドで「TMAXDB01」は、DBD作成用のスクリプト・ファイル名です。コマンドが正常に終了すると、TMAXDB01 DBDスクリプトで定義したDBD名、デフォルトDBDLIB名またはユーザー定義DBDLIB名を使用してDBD関連のメタテーブルにDBDが保存されます。

参考

1. dbdgenツール・プログラムの使用方法については、『OpenFrame ツールリファレンスガイド』を参照してください。
2. imsとhibサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。

5.3. スキーマの作成およびコピーブックの処理

メインフレームのEBCDICコードで作成されたデータをOpenFrameのASCIIコード・データに変換する場合、一般的には、OpenFrameが提供する**dsmigin** ツール・プログラムを使用します。そのと

き、データ・レコードの各フィールドの属性を定義したスキーマ・ファイルが必要です。また、各セグメントに対して表を作成するためにユーザーが使用するコピーブックを処理してメタデータを保存する必要があります。ただし、索引セグメントと可変長セグメントの場合は、dbdcpybkgenプログラムによって作成されたコピーブックを使用する必要があります。

HiDBスキーマの作成およびコピーブックの処理時には、コピーブックを分析してdsmiginツール・プログラムで入力として使用するHiDBデータのためのスキーマ・ファイルを作成し、フィールド・メタデータを保存します。そのとき、**hdgensch** ツール・プログラムを使用する必要があります。

hdgenschツール・プログラムは、**schema**コマンドを使用して各セグメントのフィールド構造を定義した1つ以上のコピーブック・ファイルを読み込んで内部的にcobgenschまたはpligenschツール・プログラムを呼び出し、作成された各スキーマ・ファイルにセグメント名ごとの条件文を追加して1つのスキーマ・ファイルとして再構成します。また、metaコマンドを使用して処理したコピーブックのフィールド情報をHiDBのメタデータ表に保存します。そのとき、allコマンドを使用すると、スキーマとメタを同時に処理することができます。

以下は、hdgenschツール・プログラムの実行コマンドの例です。

```
$ hdgensch all TMAXDB01 ./cpybook
```

上記のコマンドで「TMAXDB01」はスキーマの対象となるデータベースの登録されたDBD名であり、「./cpybook」はセグメントのフィールド情報が含まれているCOBOLコピーブックが存在するディレクトリ名です。

注意点は、ディレクトリに格納されているCOBOLコピーブックのファイル名がDBDで指定したセグメント名と同じであり、ファイルの拡張子は「.cpy」である必要があります。PL/I Copybookの場合は、hdgenschtuツールの実行時に-pオプションを指定し、ファイルの拡張子は「.inc」である必要があります。

上記のコマンドが正常に終了すると、OpenFrame環境設定のdsサブジェクトの**DATASET_DIRECTORY**セクションの**SCHEMA_DIR**キーに指定されたデフォルトのスキーマ・ディレクトリにDBD名と「.conv」拡張子で構成されたスキーマ・ファイルが作成されます。

参考

1. ツール・プログラムの使用方法については、『OpenFrame ツールリファレンスガイド』を参照してください。
 2. dsサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。
-

5.4. セグメント表の作成

次に、DBDで定義されたセグメントに対して実際にデータを保存するためのセグメント表を作成する必要があります。そのとき、**hidbmgr** ツール・プログラムを使用します。hidbmgrは、HiDBのメタデータおよびユーザー・データを管理するツール・プログラムです。segm createコマンドを使用してdbdgenで処理したDBDに対してhdgenschで処理したコピーブックのフィールド形式でセグメント表を作成します。

以下は、hidbmgrツール・プログラムの実行コマンドの例です。

```
$ hidbmgr segm create TMAXDB01
```

上記コマンドの「TMAXDB01」は、dbdgenで処理したDBD名です。

参考

hidbmgrツール・プログラムの使用方法については、『OpenFrame ツールリファレンスガイド』を参照してください。

5.5. DLIライブラリの作成

hidbmgr ツール・プログラムのdligenコマンドを使用してデータ・ロードおよびユーザー・プログラムで使用されるDLIライブラリ・ファイルを作成します。データ・ロード時には、DBDに対してのみ必要ですが、ユーザー・プログラムのHiDBコマンドが動作するには、PSBに対しても実行される必要があります。ライブラリ・ファイルは、OpenFrame環境設定の**hidb**サブジェクトの**HIDB_DEFAULT**セクションの**HIDB_OBJECT_DIR**キーに指定されたフォルダーに作成されます。

以下は、hidbmgrツール・プログラムの実行コマンドの例です。

```
$ hidbmgr dbd dligen TMAXDB01
```

上記例の「TMAXDB01」は、dbdgenで処理されたDBD名です。

参考

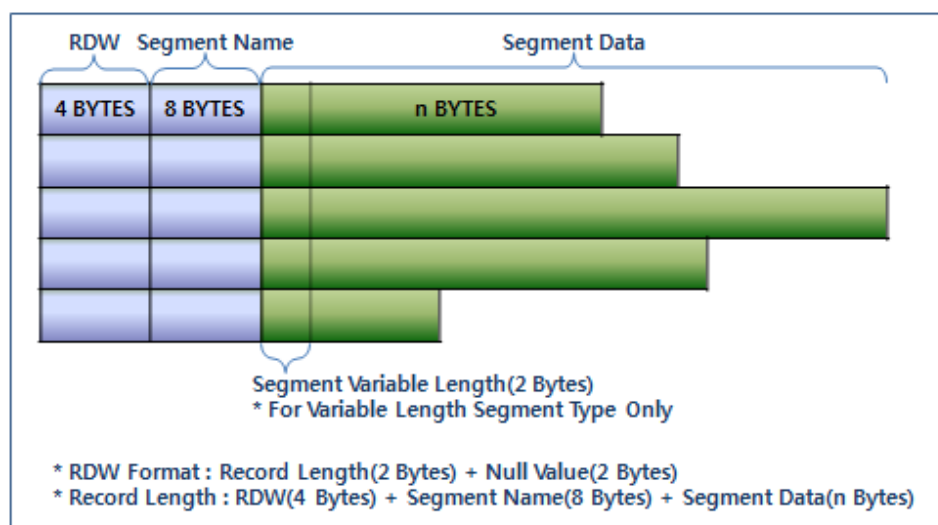
hidbmgrツール・プログラムの使用方法については、『OpenFrame ツールリファレンスガイド』を参照してください。

5.6. 標準データ・フォーマットの変換

HiDBデータベースにデータをロードするには、ロード対象ソース・データのフォーマットが事前にHiDBで指定した形式で構成されている必要があります。

HiDBで決めたデータ・フォーマットをHiDB標準データ・フォーマットといい、その形式は以下の図のとおりです。

[図 5.2] OpenFrame/HiDB標準データ・フォーマット



メインフレームのデータベースのデータが上記の形式で構成されていれば、別途の追加作業なしでHiDBのデータベースにロードできます。ただし、上記の形式で構成するためには、メインフレームで追加作業が必要な場合もあります。

IBMメインフレームのIMS/DBでは、データベースのデータをアンロードするとき、一般的にIMS/DBで提供するDFSURGU0ユーティリティを使用します。HiDBでは、DFSURGU0で作成したアンロード・データセットをHiDB標準データ・フォーマットに変換する**hdpcdf01**ツール・プログラムを提供します。したがって、DFSURGU0ユーティリティを使用してデータをアンロードした場合は、メインフレームでの追加作業は不要であり、OpenFrameで**hdpcdf01**ツール・プログラムを実行して標準データ・フォーマットに変換します。

以下は、**hdpcdf01**ツール・プログラムの実行コマンドの例です。

```
$ hdpcdf01 if=tmaxdb01.dat of=tmax01db.hdb dbd=TMAX01DB
```

上記のコマンドで「if」は入力ファイルを意味し、IBMメインフレームのIMS/DBでDFSURGU0ユーティリティを使って作成したアンロード・データセットのファイル名を指定します。「of」はHiDB標準フォーマットに変換する出力ファイル名であり、「dbd」はロード対象データベースを定義したDBD名で指定します。

参考

1. **hdpcdf01**ツール・プログラムの使用方法については、『OpenFrame ツールリファレンスガイド』を参照してください。
2. DFSURGU0ユーティリティについては、『OpenFrame ユーティリティリファレンスガイド』を参照してください。

5.7. コード変換

メインフレーム環境でのデータセットはEBCDICコードを使用し、OpenFrame環境ではASCIIコードを使用しています。したがって、メインフレームのデータセットをUNIX環境のOpenFrameデータセットにマイグレーションするときにはコード変換作業が必要です。OpenFrameではコード変換のための**dsmigin**ツール・プログラムを提供します。

dsmiginを使用するには、マイグレーション対象のソース・ファイルと該当するデータセットのフィールド別の属性情報が含まれているスキーマ・ファイルが必要です。HiDBマイグレーションの際は、HiDB標準フォーマットのソース・ファイルとhdgensschツール・プログラムを使用して作成したスキーマ・ファイルを用意します。

以下は、HiDBマイグレーション作業時のdsmiginツール・プログラムの実行コマンドの例です。

```
$ dsmigin MYDATA MYHIDBDATA -s MYSCHEMA.conv -C -f V
```

上記のコマンドにおいて、「MYDATA」はHiDB標準フォーマットで構成されたコード変換前の入力ファイル名であり、「MYHIDBDATA」はコード変換後の出力ファイル名です。「MYSCHEMA.conv」はhdgensschツール・プログラムを使用して作成したHiDBのスキーマ・ファイル名です。「-C」オプションはカタログなどの作業なしでデータコードの変換だけを行うことを意味しており、「-fV」オプションは入力ファイルの形式が可変型であることを意味します。

参考

dsmiginツール・プログラムの使用方法については、『OpenFrame ツールリファレンスガイド』を参照してください。

5.8. データのロード

この段階では、コード変換が完了したHiDB標準フォーマットのデータをHiDBにロードします。この際に使用するツール・プログラムは**hdload**です。初めてロード作業を行う場合、該当するセグメント・テーブルが空の状態であることを確認します。

以下は、hdloadツール・プログラムの実行コマンドの例です。

```
$ hdload TMAX01DB tmax01db.hdb
```

上記のコマンドで「TMAX01DB」はロード対象データベースのDBD名です。「tmax01db.hdb」はコード変換が完了したHiDB標準フォーマットのデータ・ファイル名です。

HiDBデータのロード作業中に<Ctrl>+Cキーを押して強制停止した場合、該当するデータセットに対するロックが解除されないことがあります。その状態でロード作業を再実行すると正常に動作しないため、必ず当該データセットに対するロックを解除する必要があります。

参考

hloadツール・プログラムの使用方法については、『OpenFrame ツールリファレンスガイド』を参照してください。

付録 A. DL/Iステータス・コード

本付録では、アプリケーションがDL/I関数を呼び出した後、返されるDL/Iステータス・コードについて説明します。

アプリケーションは、DL/Iステータス・コードを参照してユーザーが指示した処理を実行します。

● AC

説明	GU、GHU、GN、GHN、GNP、GHNP、ISRT、DLET、REPLなど、DL/I関数の呼び出し時に渡したSSAのエラーです。
原因	– SSAが階層レベル順に指定されていません。 – DL/IがSSAに記述したセグメント名がデータベースPCBで見つかりません。 – データベースPCBにセグメント名は記述されていますが、アクセスした階層レベル順にセグメント名が入っていません。 – 同じ階層レベルのSSAを2回記述しています。
対処方法	SSAに記述したセグメントの階層レベル順または名前を確認して修正します。

● AD

説明	関数のパラメータが有効でない場合やサポートしない関数タイプです。
原因	– 関数のパラメータが有効ではありません。 – 関数のパラメータ順が有効ではありません。 – システム・サービスの呼び出し時にI/O PCBを使用していません。 – データベースの呼び出し時にデータベースPCBを使用していません。
対処方法	記述した関数コードまたはパラメータ順が有効であるかを確認します。

● AH

説明	1つ以上のSSAパラメータが必要な関数を呼び出しました。
原因	ISRT呼び出しのときにSSAパラメータを含んでいません。
対処方法	ISRT呼び出しのときにSSAパラメータを含んでいるかを確認します。

● AI

説明	データセットを開いたときにエラーが発生しました。
原因	– 初期化ロードが実行されていないデータセットに対し、L、LS以外のDBPCBを使用した関数を呼び出しました。 – すでに初期化ロードが実行されたデータセットに対し、L、LSのDBPCBを使用してISRT関数を呼び出しました。
対処方法	データセットに初期化ロードが実行されたかを確認します。

● AJ

説明	SSAに記述した形式が有効ではありません。
原因	– SSAが有効でないコマンド・コードを含んでいます。 – 条件文の関係演算子が有効ではありません。 – 条件文に括弧文字が抜けているか、ペアになっていません。 – DLET、REPL、ISRTで修飾されたSSAを使用しています。
対処方法	SSAの形式を正しく修正します。

● AK

説明	SSAで有効でないフィールド名が見つかりました。
原因	– SSAに記述したフィールド名が正しくありません。 – SSAに記述したフィールド名がDBDに定義されていません。
対処方法	SSAのフィールド名を正しく修正します。

● AM

説明	呼び出した関数がPCBのPROCOPTに定義されているプロセッシング・オプションと適合しません。
原因	– DLET、REPL、ISRTを呼び出す場合に参照するPCBに適合するプロセッシング・オプションを記述していません。基本的にDLETはD、REPLはR、ISRTはIが設定されている必要があります。 – データベース初期化用のロード・プログラムでISRTを呼び出す場合に参照するPCBのプロセッシング・オプションをLと記述していません。
対処方法	要求した関数を修正するか、PCBの内容を修正します。

- AO

説明	TSAMで物理的なI/Oエラーが発生しました。
原因	– TSAMで障害が発生しました。 – TSAMのデータベース・ファイルが削除されたか、アクセスできません。
対処方法	TSAMの障害可否を確認し、処理します。

- AP

説明	CHKPを呼び出すときに使用したパラメータ数が有効ではありません。
原因	CHKPを呼び出すときに4つ以上のパラメータを記述しました。
対処方法	CHKPの呼び出し文を修正します。

- DA

説明	修正できないセグメントのフィールドを変更しようとしてしました。
原因	– DLETまたはREPL呼び出しでセグメントのキーを変更しようとしてしました。 – PSBのREPLオプションがNOに記述されたフィールドを修正しようとしてしました。
対処方法	要求事項を修正します。

- DJ

説明	現在hold状態でないセグメントでDLETまたはREPLを実行しました。
原因	– GHU、GHN、GHNP呼び出しで照会せず、DLETまたはREPLを実行しました。 – すでに削除されたセグメント内容でDLETを呼び出しました。
対処方法	要求事項を確認して修正します。

- DX

説明	要求したDLET呼び出しがセグメントの削除規則に違反しています。
原因	プログラムがセグメントの削除規則に違反するDLETを要求しました。
対処方法	DBAはセグメントの削除規則を確認します。

- GA

説明	上位レベルのセグメントが照会されました。
-----------	----------------------

原因	修飾されていないGNまたはGNPを呼び出した際、現在のセグメントより上位レベルのセグメントが照会されました。
対処方法	情報性のステータス・コードです。

● GB

説明	順次照会中にデータベースを最後まで読み込みました。
原因	GNを呼び出した際、データベースの最後まで照会されました。
対処方法	アプリケーション・プログラムで決定します。

● GD

説明	現在のポジションがない状態でISRTを呼び出しました。
原因	現在のポジションが削除されたか、存在しない状態でISRTを実行しました。
対処方法	アプリケーション・プログラムで決定します。

● GE

説明	条件に一致するセグメント・インスタスが見つかりません。
原因	- Get系を呼び出した際、記述した条件に一致するセグメント・インスタスが見つかりませんでした。 - ISRTを呼び出した際、挿入するセグメントの親セグメントが見つかりませんでした。
対処方法	アプリケーション・プログラムで決定します。

● GG

説明	有効でないセグメント・ポインターを持つセグメントを照会しました。
原因	- セグメント・ポインターが指し示す位置のセグメント・プレフィックス領域から有効でない形式の値が見つかりました。 - セグメントが存在していない位置を指し示すセグメント・ポインターが見つかりました。
対処方法	- データベースのデータが正常にロードされたのかを確認します。COBOLアプリケーション・プログラムを使用して正常にデータをロードするには、STOP RUNの代わりにGO BACKコマンドを使ってプログラムを終了します。 - HDAM、HIDAMデータベースの場合、1つのCIあたり1つのレコードが入れるようにレコード長を指定したのかを確認します。

● GK

説明	現在のセグメントとレベルは同じであるが、他のタイプのセグメントが照会されました。
原因	修飾されていないGNまたはGNPを呼び出した際、現在のセグメントとレベルは同じであるが、他のタイプのセグメントが照会されました。
対処方法	情報性のステータス・コードです。

● GP

説明	設定された親セグメントがありません。
原因	- 親セグメントが設定されていない状態でプログラムがGNPを要求しました。 - GNPの要求時に記述したセグメントのレベルが、設定されている親(parent)より高いです。
対処方法	GNPを呼び出す前に親セグメントを設定し、GNPのときに記述したセグメントのレベルを確認します。

● II

説明	セグメントの挿入時にデータベースに重複するセグメントが存在します。
原因	- ISRTの呼び出し時にデータベースに存在する既存のセグメントを挿入しようしました。 - 同じ物理的順次フィールドと論理的順次フィールドを持つツイン・セグメントが存在します。
対処方法	重複するセグメントの値を排除した後に処理します。

● IX

説明	要求したISRT呼び出しがセグメントの挿入規則に違反しています。
原因	- 論理子セグメントの挿入を試みた場合、それに対応する論理親セグメントまたは物理親セグメントが存在しません。 - 連結セグメントの挿入を試みた場合、物理的パスで挿入を試みた際に、論理親セグメントまたは物理親セグメントが存在しません。 - 論理的パスで挿入を試みた際に、I/O領域の論理親または物理親のキーに該当するセグメントが存在しません。
対処方法	ISRT呼び出し、あるいはプログラムを修正します。

● LB

説明	セグメントのロード時にデータベースに重複するセグメントが存在します。
原因	– 同じ物理的順次フィールドを持つツイン・セグメントが存在します。 – 物理的フォワード・ポインターを持っていないツイン・セグメントで同じタイプの2番目のセグメントを挿入しようとした。
対処方法	ISRT呼び出し、あるいはロード・コマンドを修正します。

● LC

説明	ロードされたセグメントのキーフィールドが階層的な順序から外れました。
原因	入力したデータが階層的な順序に反します。
対処方法	セグメントをチェックし、ロードする位置を決めます。

● NI

説明	一意の二次引に重複するセグメントが存在します。
原因	一意の順次フィールドである二次索引が定義されているセグメントで挿入または更新している間に、同じキーを持つ索引セグメントが見つかりました。
対処方法	業務要件に応じて決定します。重複する二次索引で発生した場合は一意でない索引を記述し、overflow entry-sequenced datasetを提供します。

● V1

説明	有効でないセグメント長が入力されました。
原因	– 可変長セグメントの場合、ISRT、REPLの呼び出し時にDBDのSEGMENT句BYTES項目に記述したセグメント長が最大値・最小値の範囲を超えて入力されました。 – ISRT、REPLの呼び出し時に入力したセグメント・データ長がデータベース・データセットのレコード長を超えました。入力したセグメント・データ長は、固定長セグメントの場合、DBDの[SEGMENT]句のBYTES項目に記述した値に従います。一方、可変長セグメントの場合は、セグメント・データI/Oバッファに記述したセグメント長に従います。また、アクセス・メソッド別のセグメント・プレフィックス長も考慮します。
対処方法	ISRT、REPL呼び出しを実行するアプリケーションを修正するか、DBDのSEGMENT句BYTES項目の値を修正します。

- ZZ

説明	HiDBの動作中に内部エラーが発生しました。
原因	HiDBのさまざまな異常動作により発生します。
対処方法	表示されるOpenFrameのエラーコードを確認して対処してください。

参考

特定のステータス・コードについての詳しい情報が必要な場合は、IBMマニュアルの『IMS V7 Application Programming: Database Manager』の「Chapter 17. DL/I Status Codes」を参照してください。

付録 B. データベース定義の固有の構文

本付録では、データベースを定義する際に使用される元のシステムと異なるOpenFrame/HiDBの固有の構文について説明します。

参考

データベースを定義するための他の構文は、元のデータベースと同じです。

B.1. SEGM文

SEGM文は、データベースにセグメント・タイプを定義するために使用されます。詳細については、[「第2章 データベースの管理」](#)の「2.2. DBD」を参照してください。

B.1.1. EXTPROC

セグメントのデータを並び替えるとき、データベースの関数またはユーザー定義の外部プロシージャを使用する場合に指定します。

- 構文

```
EXTPROC='<EP>関数形式、対象フィールド、開始位置、長さ</EP>'
```

項目	説明
<EP>、</EP>	外部プロシージャの始まりと終わりを指定します。
関数形式	適用する関数をCスタイルの文字列で指定します。プレースホルダーは%sです。
対象フィールド	プレースホルダーの対象フィールドを指定します。
開始位置	対象フィールドの開始位置を指定します。
長さ	対象フィールドの長さを指定します。

- 例

```
EXTPROC='<EP>TO_CHAR(%s), SEG1KEY, 0, 6</EP>'
```

B.1.2. GNALLPARENT

親セグメントがあるセグメントにGET NEXTを処理する際、パフォーマンス向上のためにすべての行を取得するようにSELECTクエリを実行し、スクロール可能カーソルを使用して検索するように指定します。ルート・セグメントに対しては、指定しても動作は変わりません。

- 構文

```
GNALLPARENT=( GN, GP, UNQSSA )
```

項目	説明
GN	セグメントへの検索条件を指定していないGET NEXTコマンドに適用します。
GP	セグメントへの検索条件を指定していないGET NEXT IN PARENTコマンドに適用します。
UNQSSA	unqualified検索条件を指定した場合にも、GN、GPを適用します。ただし、ルート・セグメントから指定のセグメントに至るパスで、ルート・セグメントを除くすべてのセグメントにUNQSSAとGNまたはGPが指定されている必要があります。

- 例

```
GNALLPARENT=( GN, GP )
```

索引

シンボル

/CX変数, 30

/SX変数, 30

A

ACB, 12

AIB-mask, 16

AIBマスクのフィールド, 20

APSB, 17

AREA文, 9

C

CHKP, 17

D

Data Entry Database (DEDB), 4

DATASET文, 8

DBD, 7

dbdgen, 34

DBDGEN文, 10

DBDメタデータ, 8

DBD文, 8

DL/I, 1, 15

DL/Iデータベース, 3

DL/Iデータベース・レコード, 3

DLET, 17

DLI-function, 16

DPSB, 17

dsmigin, 34, 38

E

END文, 10, 11

F

FIELD文, 9

G

GA, 19

GB, 19

Generalized Sequential Access Method (GSAM), 5

GHN, 16

GHNP, 16

GHU, 16

GK, 19

GN, 16

GNP, 16

GU, 16

H

hdgensch, 35

hdload, 38

hdpcdf01, 37

hidbmgr, 35, 36

Hierarchical Direct, 4

Hierarchical Sequential, 4

I

IMS/DB, 1

IMS/DC, 1

Index Database, 5

Index Pointer, 30

Index Source, 30

Index Target, 30

ISRT, 17

L

LCHILD文, 9

Logical Relationship, 3

P

Param-count, 16

PCB, 10

PCB-mask, 16

PCBマスクのフィールド, 18

PSB, 10

PSBGEN文, 11

Q

Qualified SSAs, 21

R

REPL, 17

ROLL, 17

S

Secondary Indices, 3

Segment-io-area, 16

Segment-search-arguments, 16

SEGM文, 9

SENFLD文, 11

SENSEG文, 11

SSA, 21

SYNC, 17

U

Unqualified SSAs, 21

X

XDFLD文, 9

XRST, 17

か

階層モデル, 2

兄弟セグメント, 3

コマンド・コード, 23

コマンド・コードの一覧, 24

さ

修飾SSA, 21

非修飾SSA, 21

システム・フィールド, 30

セグメントの階層パス, 3

セグメント・インスタンス, 2

セグメント・タイプ, 2

た

多重条件文, 26

代替PCB文, 11

ツイン・セグメント, 3

データベースPCB文, 11

データベースのロード, 7

データベースの実装, 7

データベースの維持, 7

データベースの設計, 7

ま

マイグレーション・ツールの一覧, 33

ら

連結キー, 19

論理関係, 27

論理ツイン・セグメント, 29

ルート・セグメント, 3